IVP 项目作业

俞茂桦
3210104374
信计 2101

30 May 2024

Contents

1 Introduction **3**

1.1 Story 3

1.2 编译 5

1.2.1 使用 xmake 编译 5

1.2.2 使用 make 编译 6

2 Design **7**

2.1 程序架构简介 7

2.2 程序使用简介 9

2.2.1 获取 story 数据 9

2.2.2 求解器使用 9

2.2.3 计时器使用 9

2.2.4 产生 log 9

2.2.5 绘图 10

3 Evaluation **11**

3.1 数值实验简介 11

3.2 部分数值结果展示 11

3.3 最小 cpu 时间 17

3.4 max-norm 20

3.5 不同求解器及其不同精度的求解情况 29

4 Conclusions **34**

Chapter 1

Introduction

1.1 Story

The restricted three-body system is a model for studying the motion of an earth-moon satellite or of an asteroid close enough to the earth to be strongly influenced by the earth and the sun.

In this system, the two heavy bodies are regarded as revolving in fixed orbits about their common centre of mass while the small body is attracted by the heavy bodies but never affecting their motion. To further simplify the problem, we approximate orbits of the heavy bodies as circles so that we can work in a frame of reference that rotates with the two heavy bodies. In this frame, the two heavy bodies are considered as fixed in space with their rotation translated into a modification of the equations of gravitational motion. For simplicity, we scale the units to reduce a number of constants to one. We also scale the masses of the two heavy bodies to $1-\mu$ and µ and their positions relative to the moving reference frame by the vectors μe_1 and $(\mu-1)e_1$ respectively so that their center of mass is at the origin of coordinates.

Denote by u_1, u_2 and u_3 the scalar variables representing the position coordinates of the small body and by u_4, u_5 and u_6 the corresponding velocity coordinates. The equations of motion for u(t) are

$$\begin{cases} u_1' = u_4 \\ u_2' = u_5 \\ u_3' = u_6 \\ u_4' = 2u_5 + u_1 - \frac{\mu(u_1+\mu-1)}{(u_2^2+u_3^2+(u_1+\mu-1)^2)^{3/2}} - \frac{(1-\mu)(u_1+\mu)}{(u_2^2+u_3^2+(u_1+\mu)^2)^{3/2}} \\ u_5' = -2u_4 + u_2 - \frac{\mu u_2}{(u_2^2+u_3^2+(u_1+\mu)^2)^{3/2}} - \frac{(1-\mu)u_2}{(u_2^2+u_3^2+(u_1+\mu)^2)^{3/2}} \\ u_6' = -\frac{\mu u_3}{(u_2^2+u_3^2+(u_1+\mu-1)^2)^{3/2}} - \frac{(1-\mu)u_3}{(u_2^2+u_3^2+(u_1+\mu)^2)^{3/2}} \end{cases}$$

Planar motion is possible; that is, solutions which satisfy $u_3 = u_6 = 0$ at all times. One of these is shown in the LHS plot of Figure 10.14, with the values of (u_1, u_2) plotted as the orbit evolves. The heavier mass is at the point $(\mu, 0)$ and the lighter mass is at $(1-\mu, 0)$, where $(0, 0)$ is marked 0 and (1, 0) is marked 1. For this calculation the value of $\mu = 0.012277471$ was selected, corresponding to the earth-moon system. The initial values for this computation were

$$u(0) = (0.994, 0, 0, 0, -2.0015851063790825224, 0) \qquad (10.190)$$

and the period was $T_1 = 17.06521656015796$.

The RHS plot of Figure 10.14 illustrates another solution of (10.189) with a different initial condition

$$u(0) = (0.879779227778, 0, 0, 0, 0.379677780949, 0) \qquad (10.191)$$

and a different period $T_2 = 19.140540691377$.

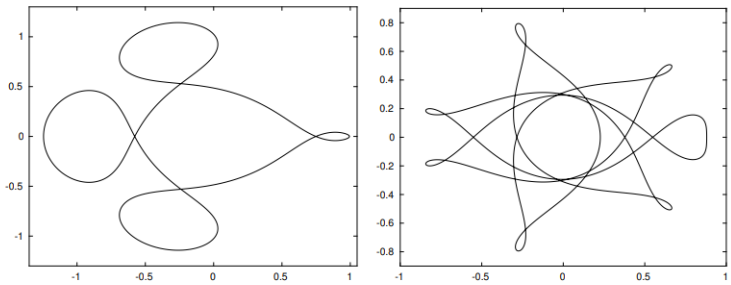


Figure 10.14: Planar orbits of the three-body ODE system (10.189) with initial conditions (10.190) and (10.191).

1.2 编译

注: 由于在确定满足误差小于一定值的最小 cpu 时间时，采用了二分搜索法，以及控制自适应步长算法的总步数时采用了遍历搜索方法，导致运行完整的程序需要的时间较长，建议注释一部分，运行一部分。在自行测试代码时，如果不使用求解器工厂获得求解器，请注意使用智能指针以便及时释放内存。

注:windows 下编译的 report 后面的图片会更大, 也有可能是受使用的编译器的影响.

1.2.1 使用 xmake 编译

```
$ xmake build ivp //生成静态库
$ xmake build test //编译测试程序
$ xmake run test //运行测试程序
$ xmake run plot //将测试程序的输出绘制成report需要的图片
$ xmake build report //编译latex文件程序报告
$ xmake clean //清除所有生成的文件
$ xmake build all //按顺序运行以上除了clean的所有
```

1.2.2 使用 make 编译

```
$ make ivp //生成静态库
$ make test //编译测试程序
$ make run_test //运行测试程序
$ make run_plot //将测试程序的输出绘制成report需要的图片
$ make report //编译latex文件程序报告
$ make clean //清除所有生成的文件
$ make all/make //按顺序运行以上除了clean的所有
```

Chapter 2

Design

2.1 程序架构简介

在程序设计上, 主要实现了以下几个 class 及成员函数.

- TimeIntegrator 类: 所有 story 中微分方程的求解器的基类

  virtual initialize：主要是根据阶数或者 state 数 (Gauss Legendre RK Methods) 来初始化参数

  virtual solve：求解函数，第一个参数是初值，第二个参数是 $u(t)' = f(u,t)$ 的右侧函数，第三个参数是 stepnumber(在自适应步长的方法中实际上只决定第一步的步长)，第四个参数是开始时间，第五个参数是结束时间

  output：将求解结果输出到文件，第一个参数输出文件路径及文件名，不存在的文件夹会被自动创建，第二个参数是输出后是否清除储存的解 (减小内存消耗) 的 bool 变量，第三个参数是输出到文件的形式为添加还是覆写 (app 还是 out)

- TimeIntegratorFactory 类: 管理求解器的工厂，是一个单例的类

  registerTimeIntegrator：注册求解器，第一个参数为求解器名，第二个参数是返回指向对应求解器类的 unique_ptr<TimeIntegrator> 的右侧函数

  createTimeIntegrator：根据求解器名创建求解器，第一个参数为求解器名，第二个参数为阶数（主要对于 Gauss Legendre RK 方法之外的固定步

长方法）或者 state 数（对于 Gauss Legendre RK 方法），自适应步长方法和 Euler 方法随意

getInstance：获取求解器工厂的唯一实例

- TimeIntegrator 的派生类

  Adama_Bashforth_Methods：对应求解器名 Adams-Bashforth

  Adams_Moulton_Methods：对应求解器名 Adams-Moulton

  Backward_Differentiation_Formula：对应求解器名 Backward_Differentiation

  ClassicalRK：对应求解器名 ClassicalRK

  ESDIRK：对应求解器名 ESDIRK

  GLRK：对应求解器名 GLRK

  FhelbergRK：对应求解器名 FehlbergRK

  Dormand_PrinceRK：对应求解器名 Dormand-PrinceRK

  Euler_Meythod：对应求解器名 Euler

- testsolver 函数：测试求解器在 story 问题下的求解，对于固定步长方法，使用二分法寻找 10000000 步内的使得误差的 max-norm 小于 0.001 的最小 cputime，对于所有方法的所有阶数的求解器，在步数为 1000，10000，100000，1000000 步情况下求解 story 中问题（对于自适应步长方法，实际上只有初始步长受影响）。第一个参数为求解器名，第二、三个参数为初始化时 order 或 state number 的最小值和最大值。

- outputcapture 函数：获取输出到终端的内容并输出到 txt 文件内，第一个参数是 txt 文件地址，为创建的文件夹会自动创建，第二、三、四个参数为 testsolver 的参数，第三个参数为函数（实际上输入 testsolver）。

- UnitTimer 类：给程序运行的 cpu 时间记时

  begin: 第一个参数为 task 名称，开始这个 task 的计时

  end: 第一个参数为 task 名称，结束这个 task 的计时

  getTime: 第一个参数为 task 名称，输出该 task 的 cpu 时间

  reset：清除所有计时器

  report：在终端打印所有 task 的时间以及程序运行总时间

getInstance：获取计时器的唯一实例

- VectorFunction 类：向量函数类，有重载括号运算符的虚函数，其第一个参数是 $u'(t) = f(u,t)$ 中的 $u(std :: vector < double >)$, 第二个参数是 $t(double)$; 对于某些求解器，需要正确设定其 dim 值 (即 u 的维数)。

2.2 程序使用简介

2.2.1 获取 story 数据

```
StoryFunction storyfunc;
const VectorFunction* func = &storyfunc;
```

注意 StoryFunction 已经定义在 VectorFunction.h 中，表示 story 中的 $f(u,t)$。

2.2.2 求解器使用

```
std::unique_ptr<TimeIntegrator> solver = TimeIntegratorFactory::getInstance()
.createTimeIntegrator("classicalRK", 4);\\创建求解器
solver->solve(initial_values,func,100000,0.0,T_1);\\求解
solver->output("output/.../ClassicalRK_100000.txt");\\输出
```

2.2.3 计时器使用

```
UnitTimer& timer=UnitTimer::getInstance();\\获取计时器的唯一实例
timer.reset();\\清空计时器
timer.begin("task1");\\开始计时
timer.end("task1");\\结束这个task的计时
std::cout<<timer.getTime("task1")<<std::endl\\输出该task的cpu时间
timer.report();\\在终端打印所有task的时间以及程序运行总时间
```

2.2.4 产生 log

```
outputcapture("../log/Adams-Bashforth.txt",func,...(args of func));
```

2.2.5 绘图

```
python plot/plot.py
#将output/track文件夹下每个最深层子文件夹的数据绘成一幅图
#将output/acc文件夹下每个文件绘成一幅图
```

img/acc 下每一个文件储存了每个求解器不同阶数在 1000,10000,100000,1000000 作为 step 数时求解的误差的最大范数，绘图逻辑是将 1000 的误差除以 10000 的误差然后对 10 取对数获得一个点，对于其它情况类似。然后将每阶的对应的点连成一条折线。

Chapter 3

Evaluation

3.1 数值实验简介

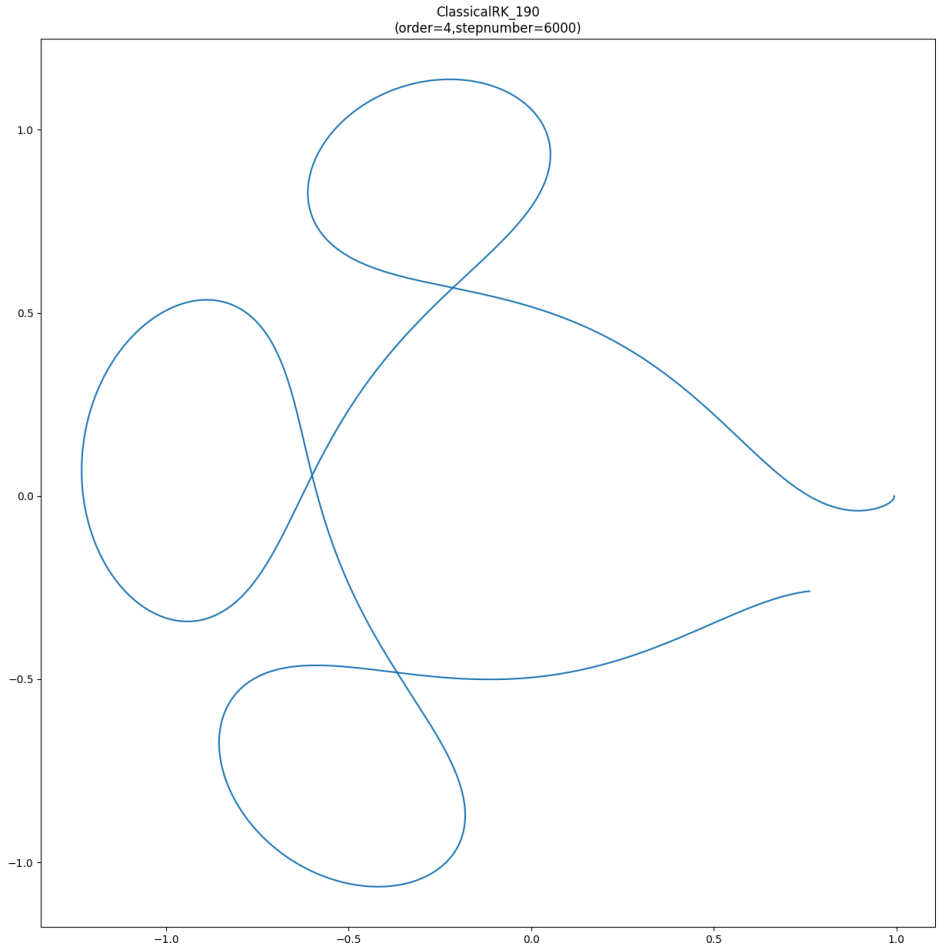
测试了对于 story 中问题的两个初始状态，每个求解器在输入步数为 1000，10000，100000，1000000 的情况下的不同阶数的求解情况，使用二分法寻找对于固定步长的求解器在 10000000 步数内的最小 cpu 时间。

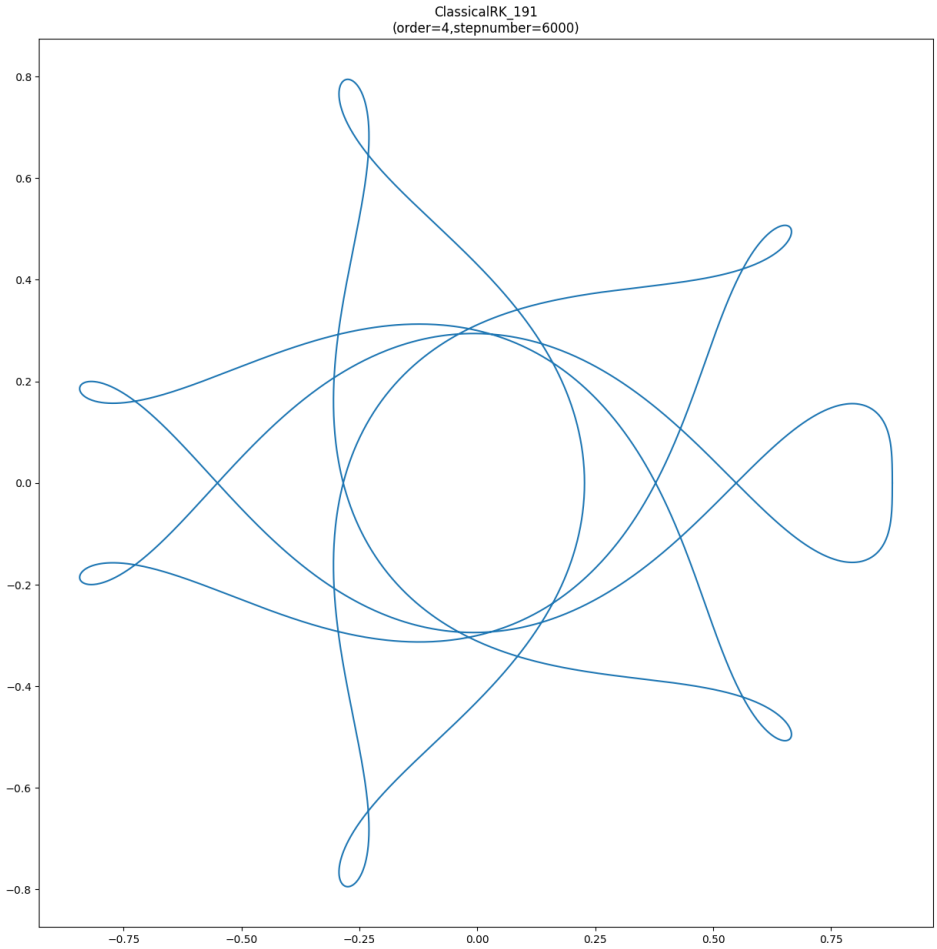
- 求解情况在 output/track 文件夹下，每一行为一个点的坐标。
- 对应的 maxnorm 在 output/acc 文件夹下，每一行为某阶某求解器在某初值下的误差最大范数和 cpu 时间。

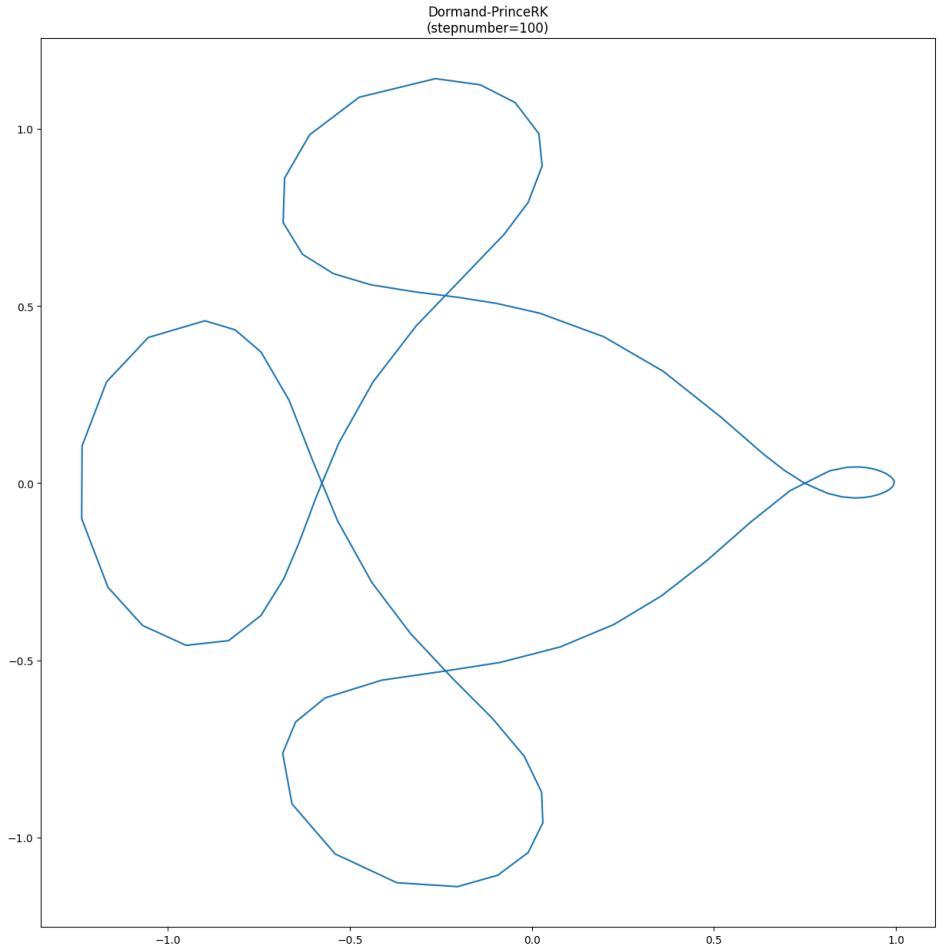
  对于 10.191 的情况，求误差方法为用同样的求解器求解 0 到 T2/2 的解，将其在 T2/2 的值作为初值，求解 T2/2 到 T2 的解，然后利用理查德外推法的公式修正最终解，使用该解与原解计算误差。
- 对应最小 cpu 时间在 output/mincpu 文件夹下，每一行为某求解器在某初值下的最小 cpu 时间（对于自适应步长并不生效）。
- 对应 log 在 log 文件夹下，为命令行输出内容。

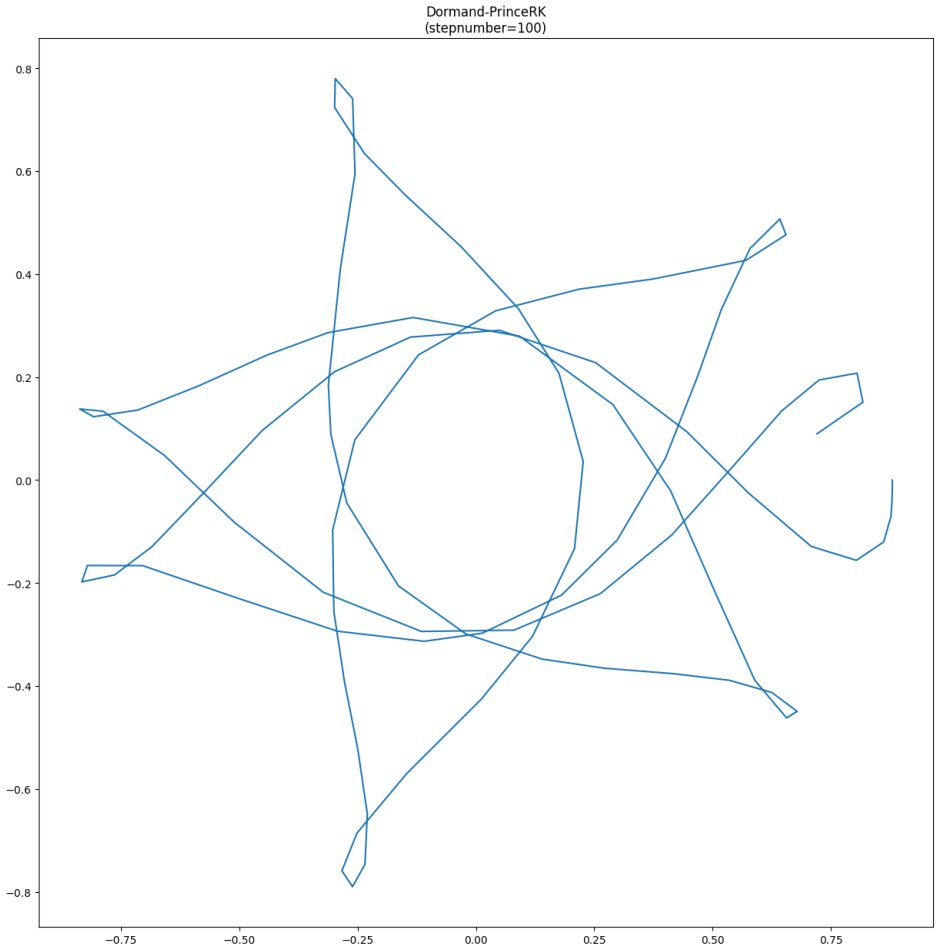
3.2 部分数值结果展示

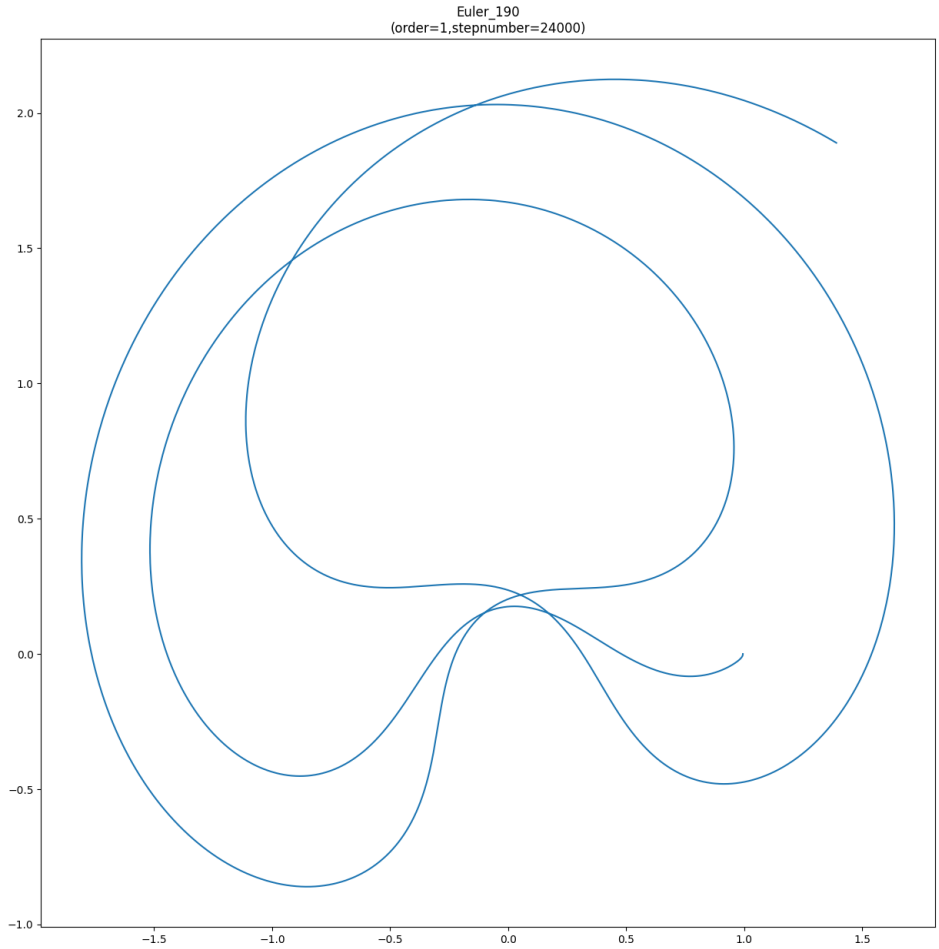
作业中要求的数据生成在 output/track1 和 output/acc1 中，其结果如下

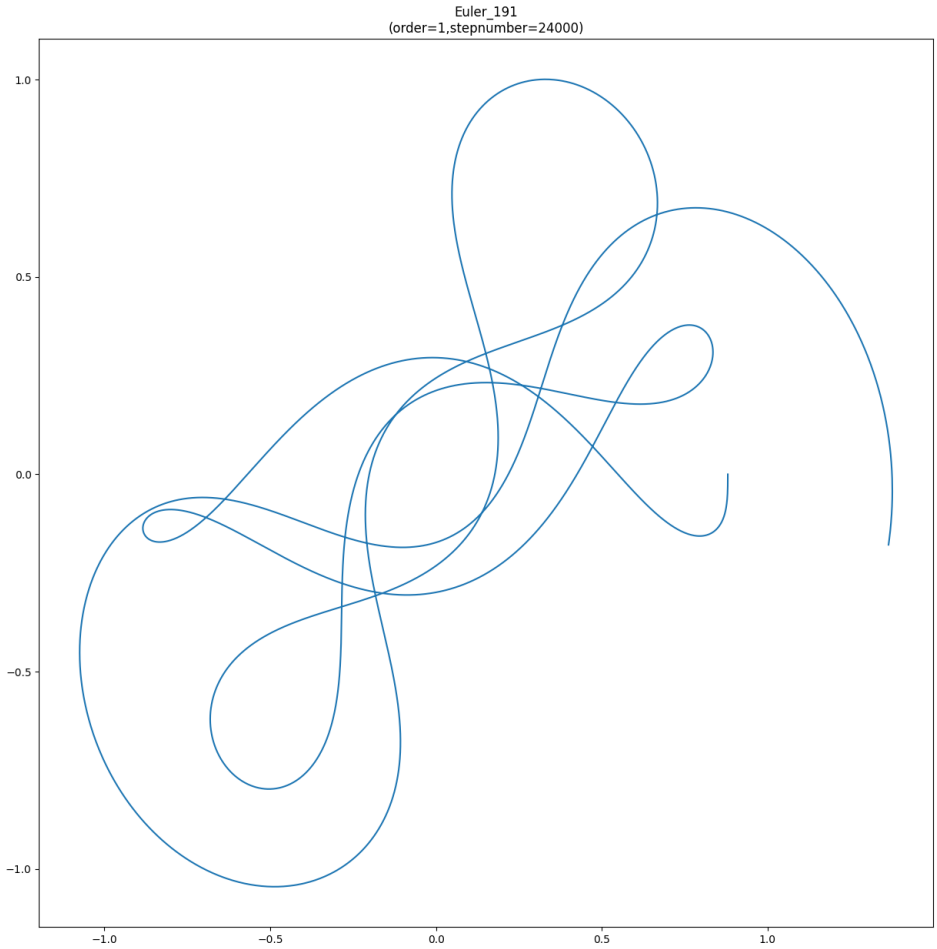

ClassicalRK_190
(order=4,stepnumber=6000)
1.0
0.5
0.0
−0.5
−1.0
−1.0
−0.5
0.0
0.5
1.0


ClassicalRK_191
(order=4,stepnumber=6000)
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4
−0.6
−0.8
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75


Dormand-PrinceRK
(stepnumber=100)
1.0
0.5
0.0
−0.5
−1.0
−1.0
−0.5
0.0
0.5
1.0


Dormand-PrinceRK
(stepnumber=100)
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4
−0.6
−0.8
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75


Euler_190
(order=1,stepnumber=24000)
2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5



3.3 最小 cpu 时间

使用二分法寻找了使得误差小于 0.001 的最小 cpu 时间 (最小步数)，结果如下 (第一个数据为步数，第二个数据为误差最大范数，第三个数据为 cpu 时间，为-1,0,0 表示没有在 10000000 步内寻找到):

- Adams-Bashforth_190 (order = 1) : -1 0 0
- Adams-Bashforth_190 (order = 2) : -1 0 0
- Adams-Bashforth_190 (order = 3) : 790037 0.000999998 4.2921
- Adams-Bashforth_190 (order = 4) : 406561 0.000999995 2.52626

- Adams-Bashforth_191 (order = 1) : -1 0 0
- Adams-Bashforth_191 (order = 2) : 23895 0.000999962 0.0838745
- Adams-Bashforth_191 (order = 3) : 16077 0.000999947 0.0832476
- Adams-Bashforth_191 (order = 4) : 3292 0.000999077 0.0254242

- Adams-Moulton_190 (order = 2) : -1 0 0
- Adams-Moulton_190 (order = 3) : 375851 0.000999999 4.18314
- Adams-Moulton_190 (order = 4) : 212824 0.000999984 3.03074
- Adams-Moulton_190 (order = 5) : 92293 0.000999971 1.26825
- Adams-Moulton_191 (order = 2) : 11902 0.000999938 0.183787
- Adams-Moulton_191 (order = 3) : 7743 0.000999873 0.126579
- Adams-Moulton_191 (order = 4) : 1715 0.000997934 0.0400893
- Adams-Moulton_191 (order = 5) : 1750 0.000997853 0.0480614

- Backward Differentiation_190 (order = 1) : -1 0 0
- Backward Differentiation_190 (order = 2) : -1 0 0
- Backward Differentiation_190 (order = 3) : 681187 0.000999993 7.17079
- Backward Differentiation_190 (order = 4) : 352936 0.000999999 3.63705
- Backward Differentiation_191 (order = 1) : -1 0 0
- Backward Differentiation_191 (order = 2) : 20144 0.000999938 0.342755
- Backward Differentiation_191 (order = 3) : 14073 0.00099985 0.20502
- Backward Differentiation_191 (order = 4) : 3015 0.000998607 0.102383

- ClassicalRK_190 : 85645 0.00099996 0.811399

- ClassicalRK_191：1080 0.000998308 0.0087803

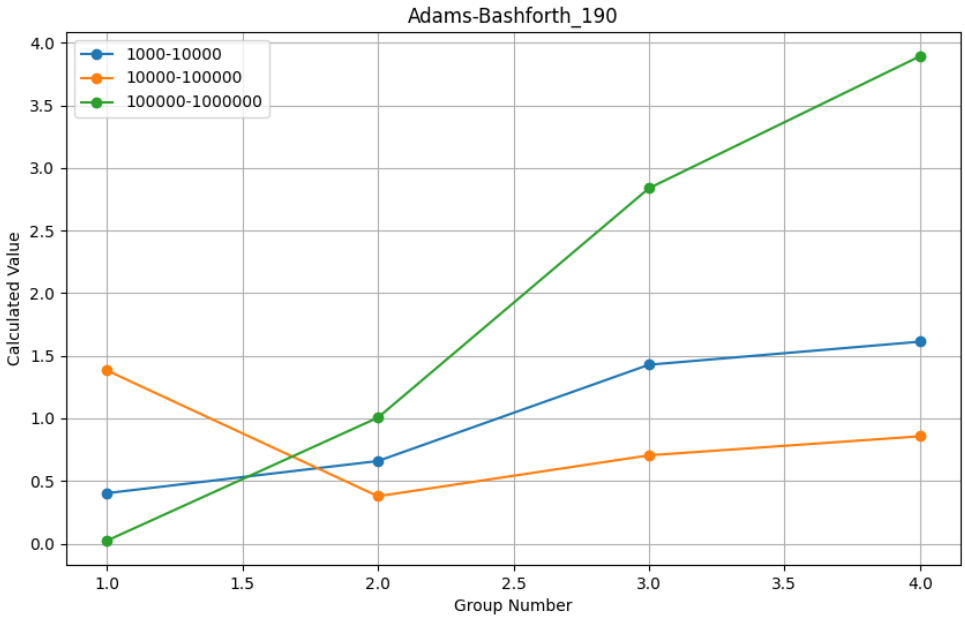
- ESDIRK_190：66138 0.000999983 6.72071
- ESDIRK_191：669 0.000998584 0.122715

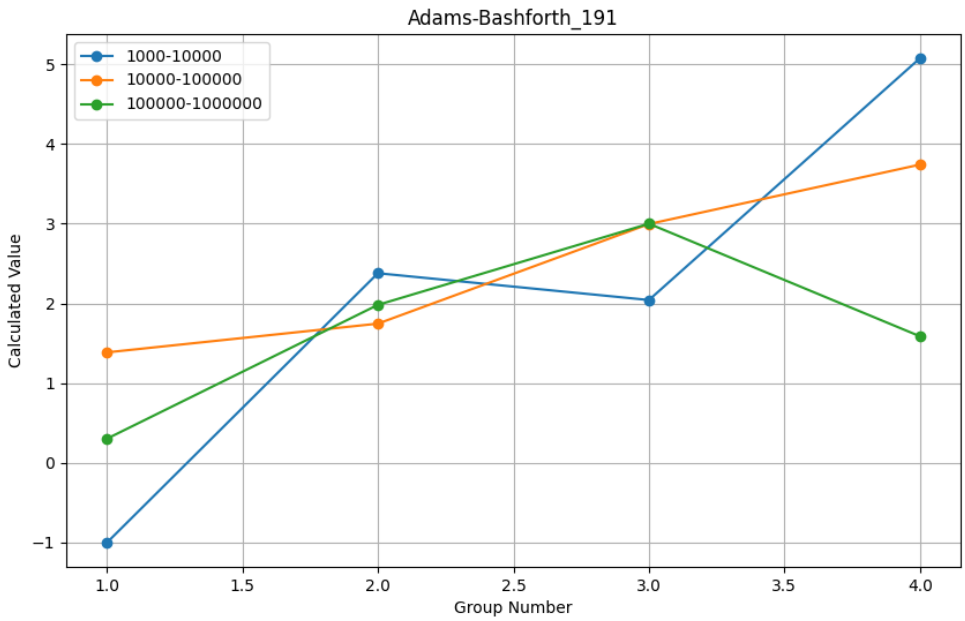
- GLRK_190 (order = 2)：59705 0.000999987 2.09739
- GLRK_190 (order = 3)：17183 0.000999848 0.962451
- GLRK_190 (order = 4)：7470 0.000999249 0.832366
- GLRK_190 (order = 5)：4942 0.000998519 1.08603
- GLRK_190 (order = 2)：631 0.000995117 0.211923
- GLRK_190 (order = 2)：222 0.000452825 0.16272
- GLRK_190 (order = 2)：125 0.000304964 1.18271
- GLRK_190 (order = 2)：94 0.000654174 0.921698

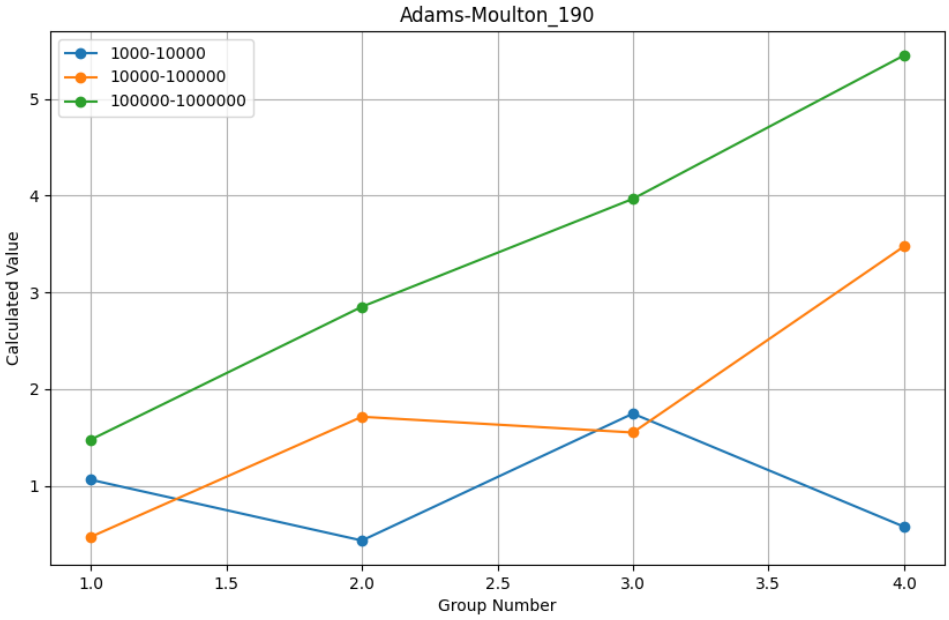
- Euler_190：-1 0 0
- Euler_191：-1 0 0

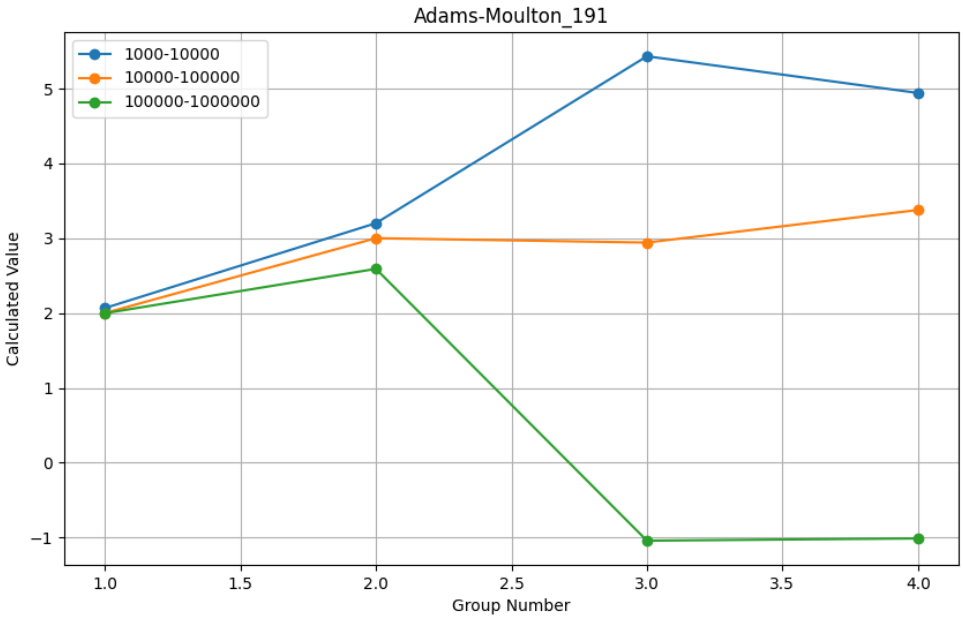
对于自适应步长方法，还没有设计相应的搜索误差小于 0.001 的最小 cpu 时间，观察 log 发现第五个分量比较难收敛，但是由于其自适应步长的特性，不论初始步长是什么，计算的时间都差不多，而且很快，从结果来看准确率也不错，不过在我默认的参数（Eabs，Erel，ρ，ρ_{max}，ρ_{min}）下第五个分量的误差比较难收敛于 0。

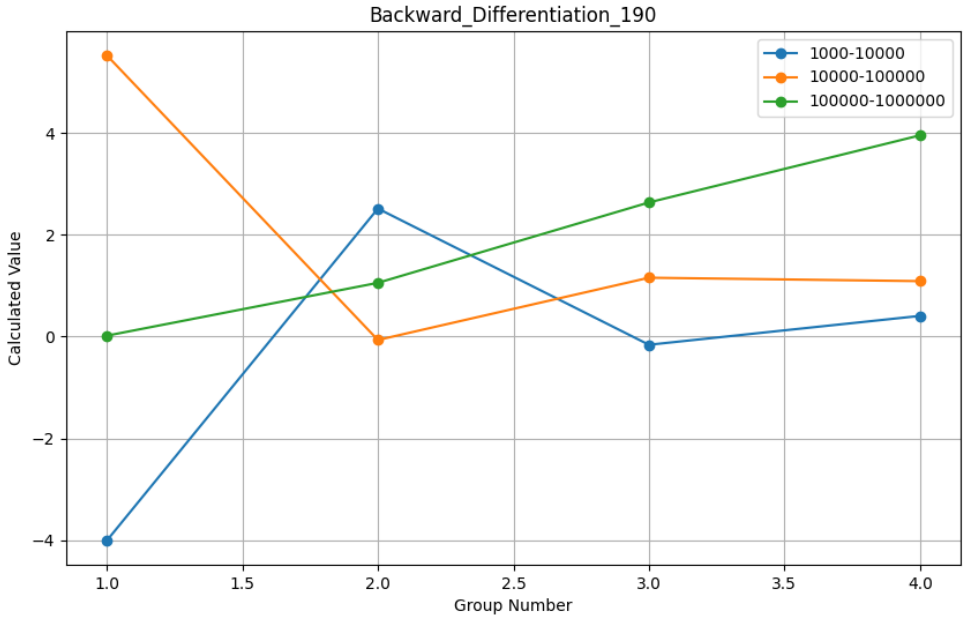
3.4 max-norm

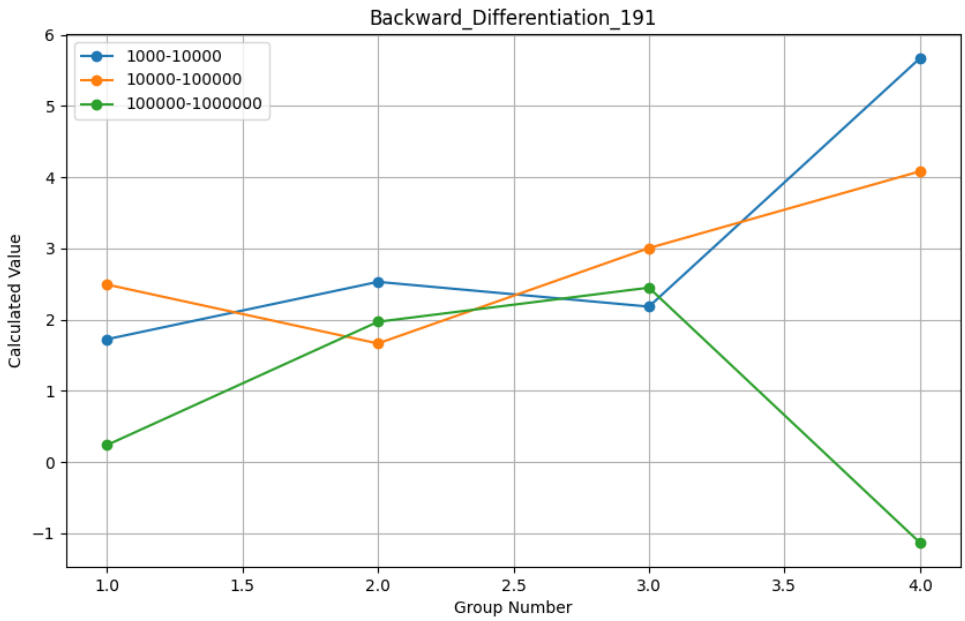

Adams-Bashforth_190
1000-10000
10000-100000
100000-1000000
Calculated Value
Group Number
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
1.0
1.5
2.0
2.5
3.0
3.5
4.0

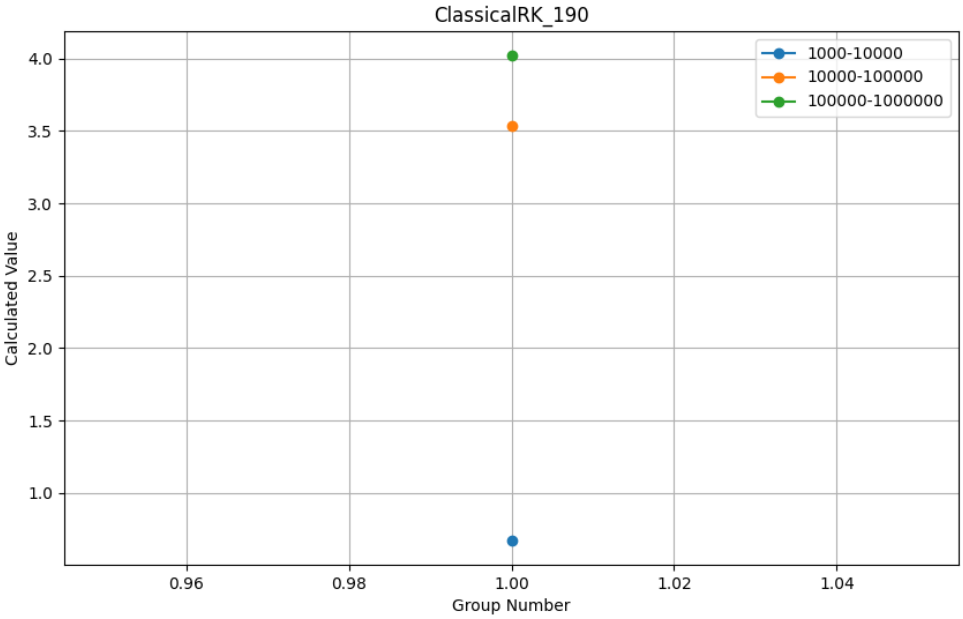

Adams-Bashforth_191
1000-10000
10000-100000
100000-1000000
Calculated Value
Group Number
5
4
3
2
1
0
−1
1.0
1.5
2.0
2.5
3.0
3.5
4.0

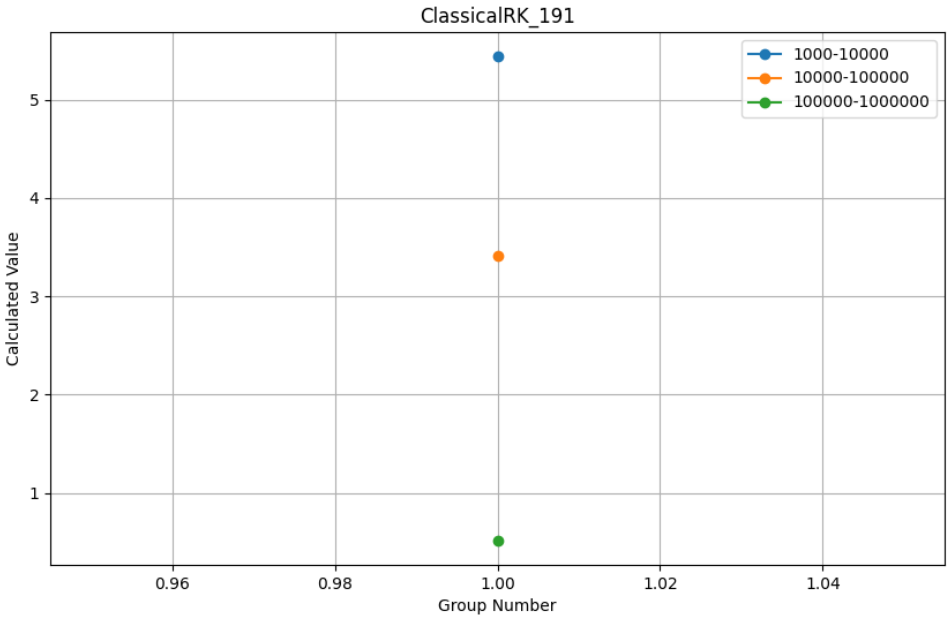

Adams-Moulton_190
1000-10000
10000-100000
100000-1000000
Calculated Value
Group Number
1.0
1.5
2.0
2.5
3.0
3.5
4.0
1
2
3
4
5

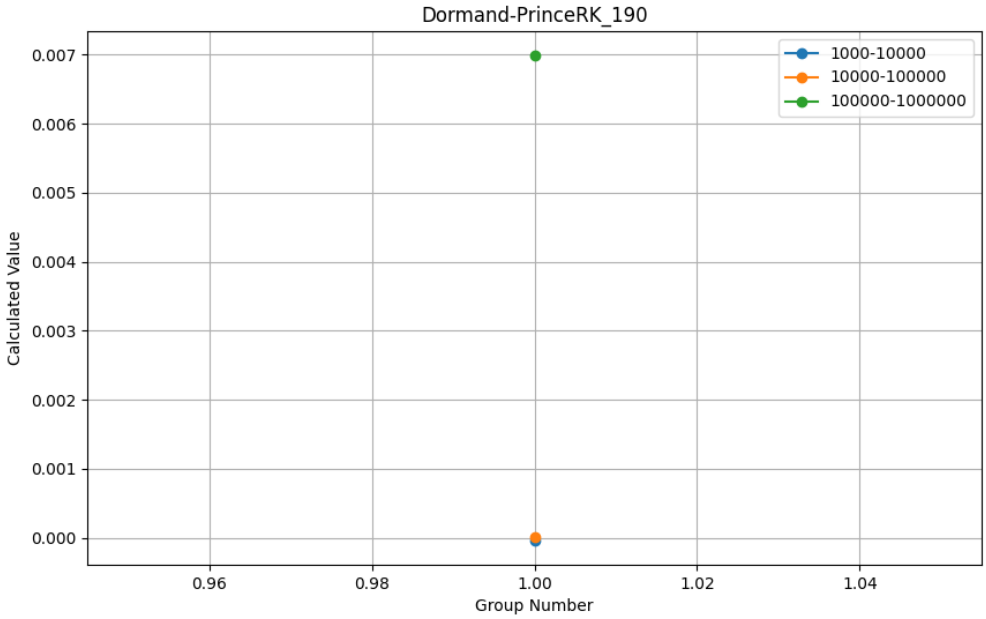

Adams-Moulton_191
1000-10000
10000-100000
100000-1000000
Calculated Value
Group Number
1.0
1.5
2.0
2.5
3.0
3.5
4.0
−1
0
1
2
3
4
5

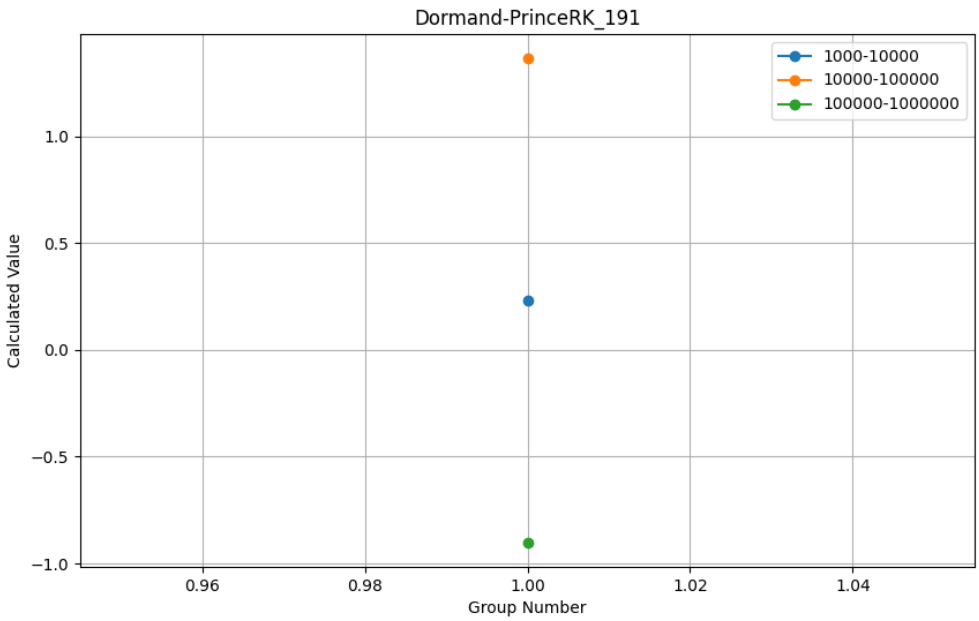

Backward_Differentiation_190
1000-10000
10000-100000
100000-1000000
Calculated Value
Group Number
4
2
0
−2
−4
1.0
1.5
2.0
2.5
3.0
3.5
4.0

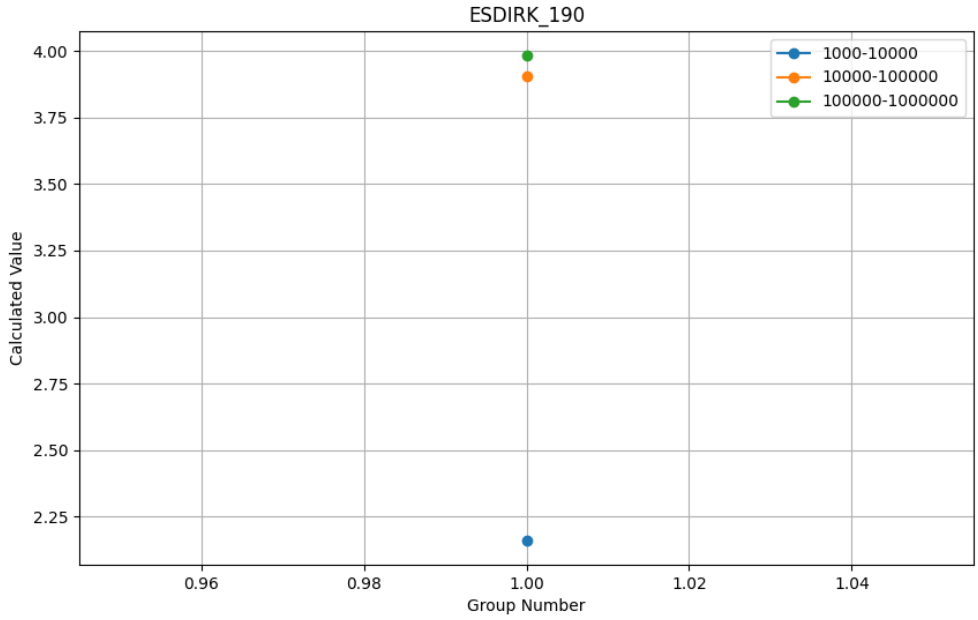

Backward_Differentiation_191
1000-10000
10000-100000
100000-1000000
Calculated Value
Group Number
6
5
4
3
2
1
0
−1
1.0
1.5
2.0
2.5
3.0
3.5
4.0

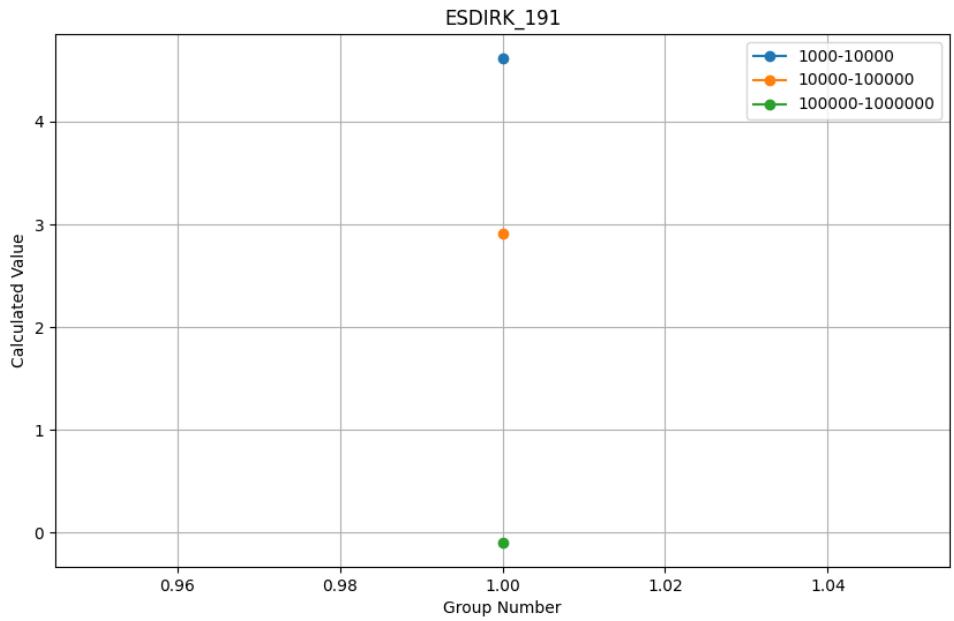

ClassicalRK_190
1000-10000
10000-100000
100000-1000000
Calculated Value
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.96
0.98
1.00
1.02
1.04
Group Number

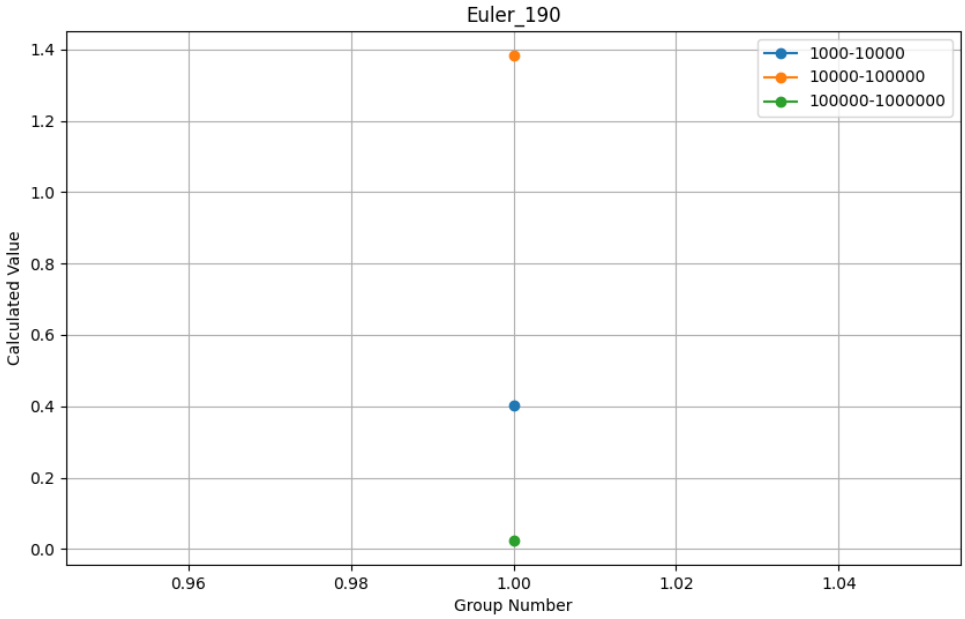

ClassicalRK_191
1000-10000
10000-100000
100000-1000000
Calculated Value
5
4
3
2
1
0.96
0.98
1.00
1.02
1.04
Group Number

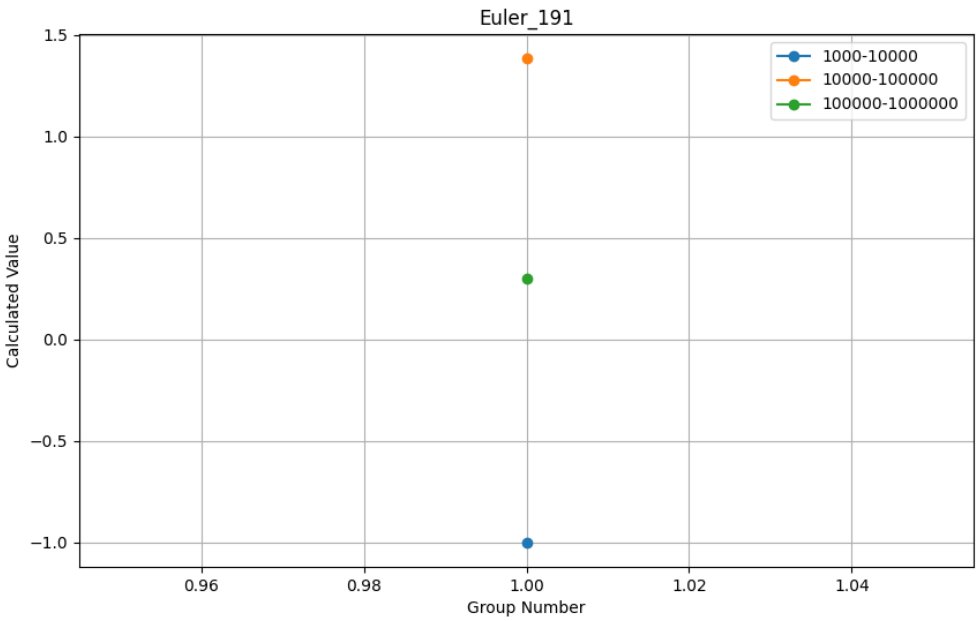

Dormand-PrinceRK_190
1000-10000
10000-100000
100000-1000000
Calculated Value
0.007
0.006
0.005
0.004
0.003
0.002
0.001
0.000
0.96
0.98
1.00
1.02
1.04
Group Number

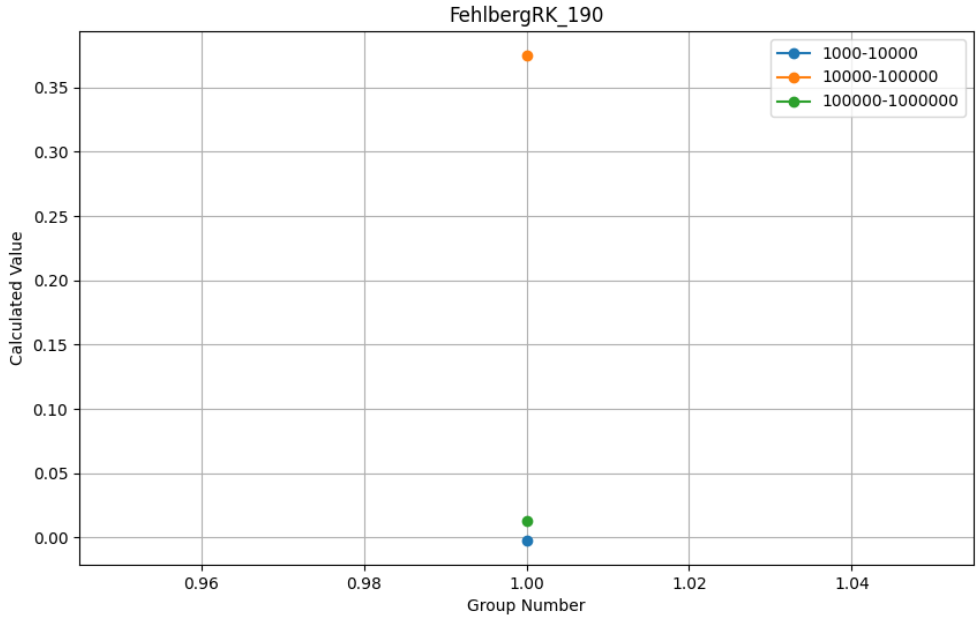

Dormand-PrinceRK_191
1000-10000
10000-100000
100000-1000000
Calculated Value
1.0
0.5
0.0
−0.5
−1.0
0.96
0.98
1.00
1.02
1.04
Group Number

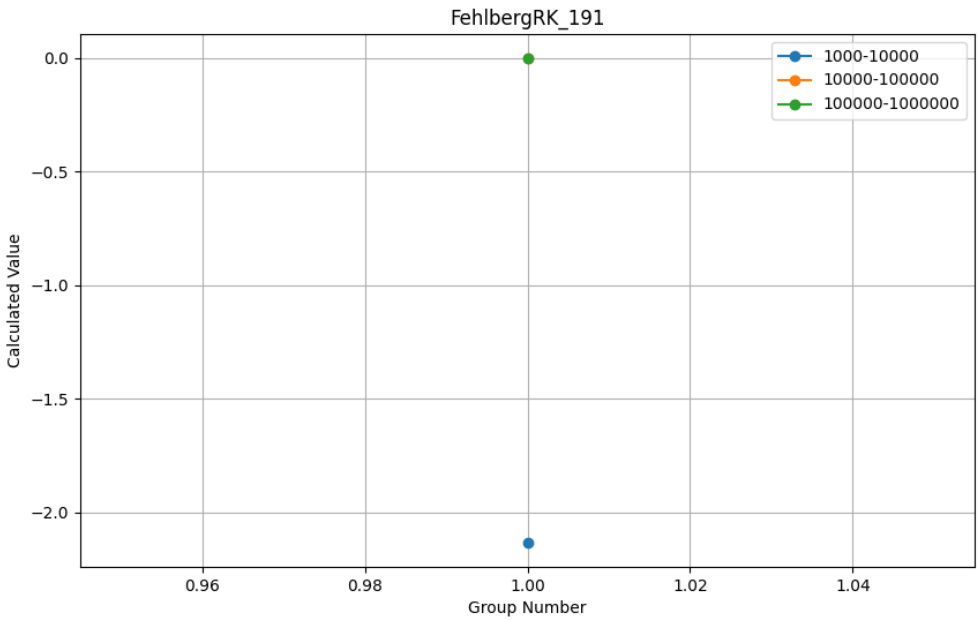

ESDIRK_190
1000-10000
10000-100000
100000-1000000
Calculated Value
4.00
3.75
3.50
3.25
3.00
2.75
2.50
2.25
0.96
0.98
1.00
1.02
1.04
Group Number

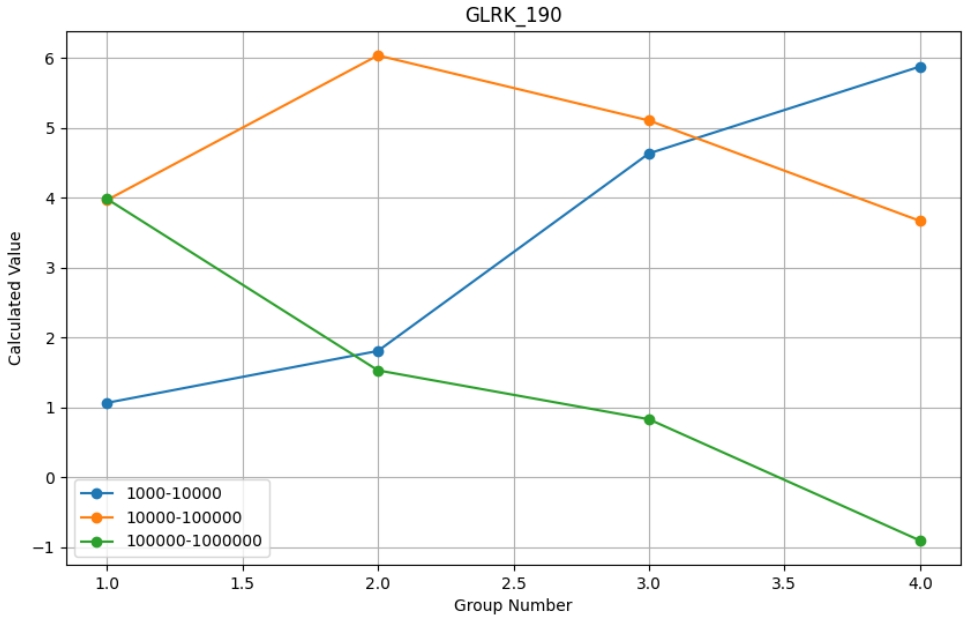

ESDIRK_191
1000-10000
10000-100000
100000-1000000
Calculated Value
4
3
2
1
0
0.96
0.98
1.00
1.02
1.04
Group Number

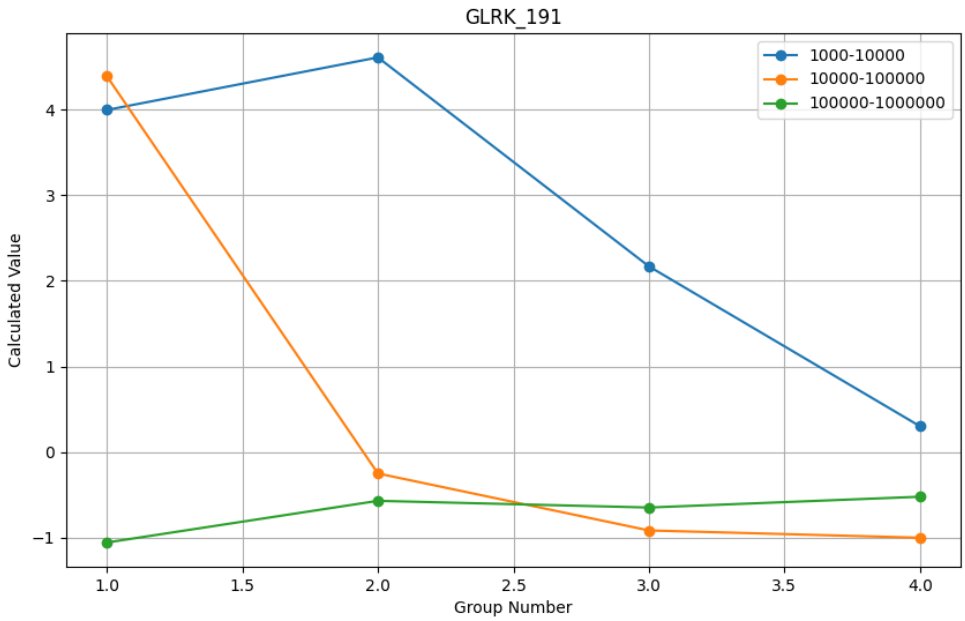

Euler_190
1000-10000
10000-100000
100000-1000000
Calculated Value
Group Number
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
0.96
0.98
1.00
1.02
1.04


Euler_191
1000-10000
10000-100000
100000-1000000
Calculated Value
Group Number
1.5
1.0
0.5
0.0
−0.5
−1.0
0.96
0.98
1.00
1.02
1.04


FehlbergRK_190
1000-10000
10000-100000
100000-1000000
Calculated Value
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00
0.96
0.98
1.00
1.02
1.04
Group Number


FehlbergRK_191
1000-10000
10000-100000
100000-1000000
Calculated Value
0.0
−0.5
−1.0
−1.5
−2.0
0.96
0.98
1.00
1.02
1.04
Group Number

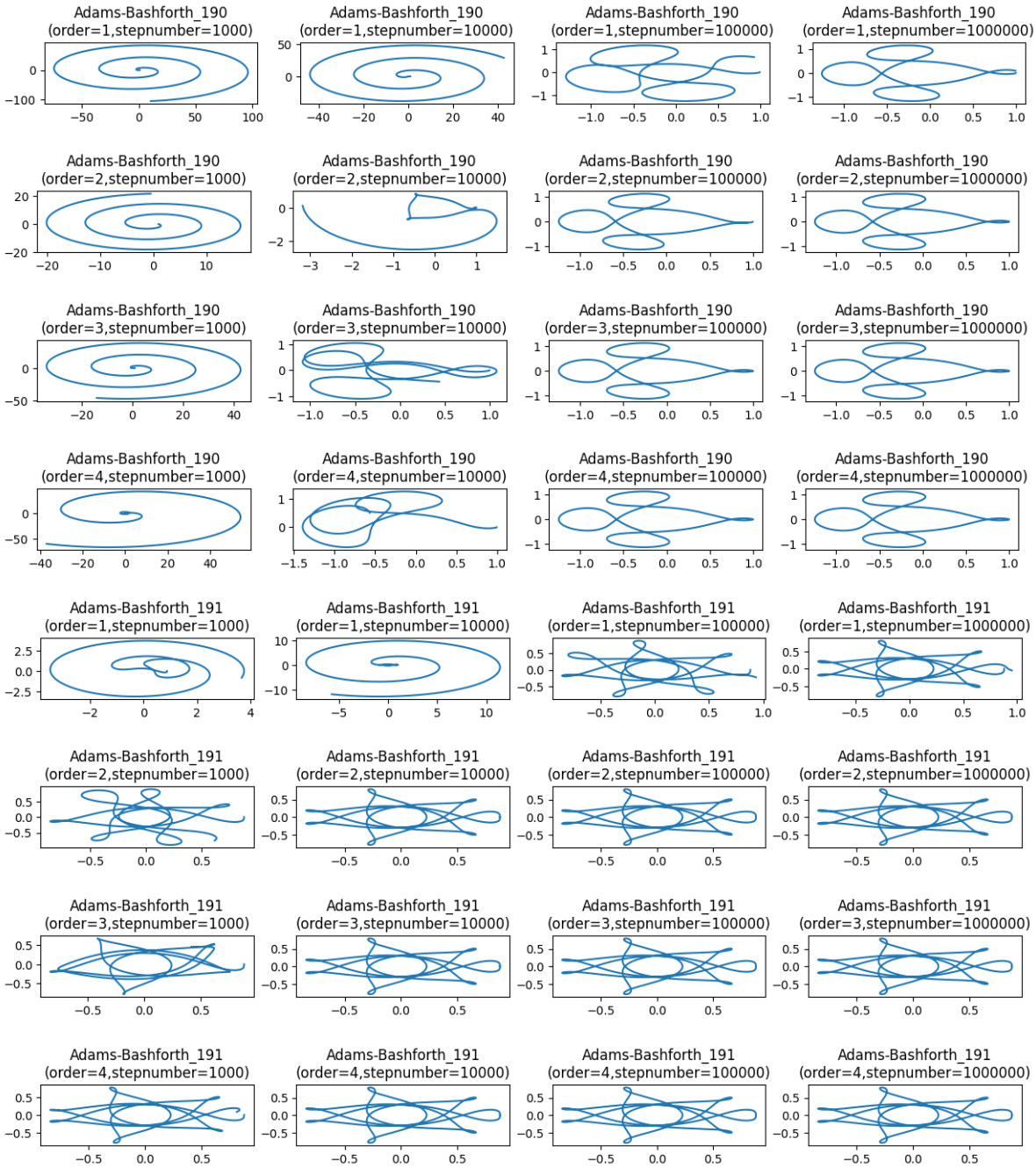


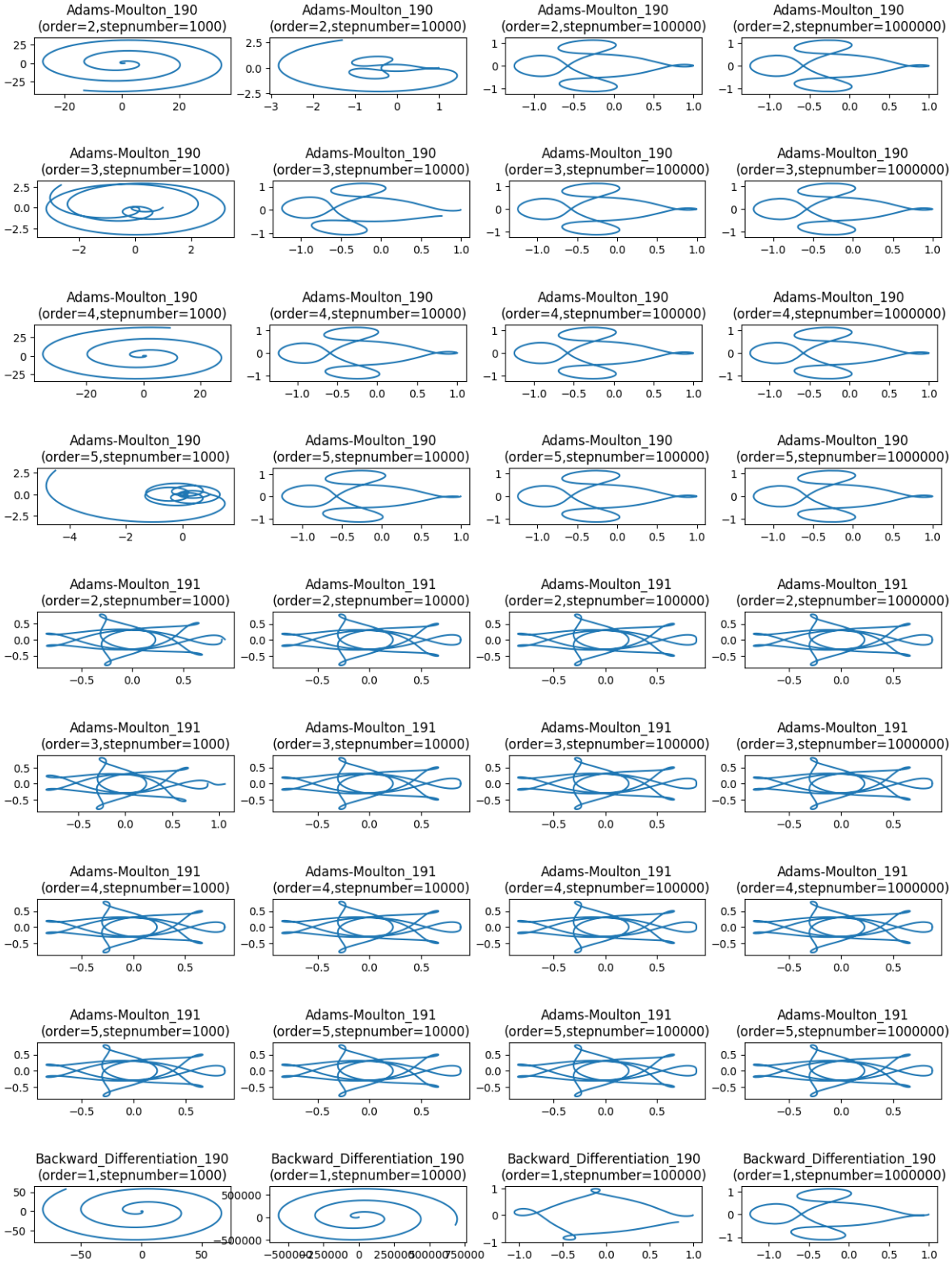


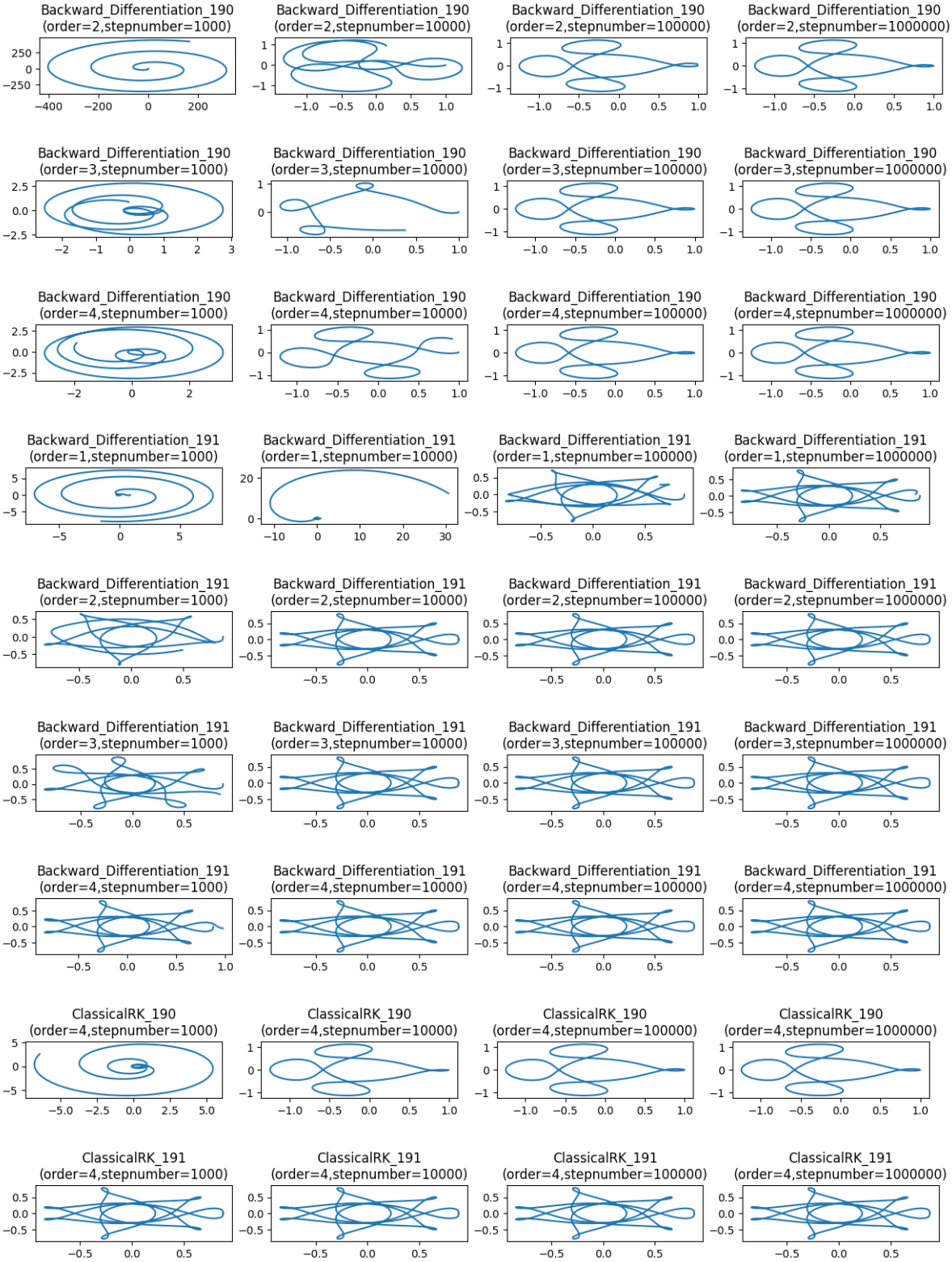
观察 log 发现对固定步长，误差接近及其精度时折现表现得不太符合数据（即点应该在对应阶数附近）。但是在误差较大时，是比较符合的，所有固定步长的方法，都可以找出一条符合算法阶数的折线（除了 GLRK 求解 10.191 的情况，因

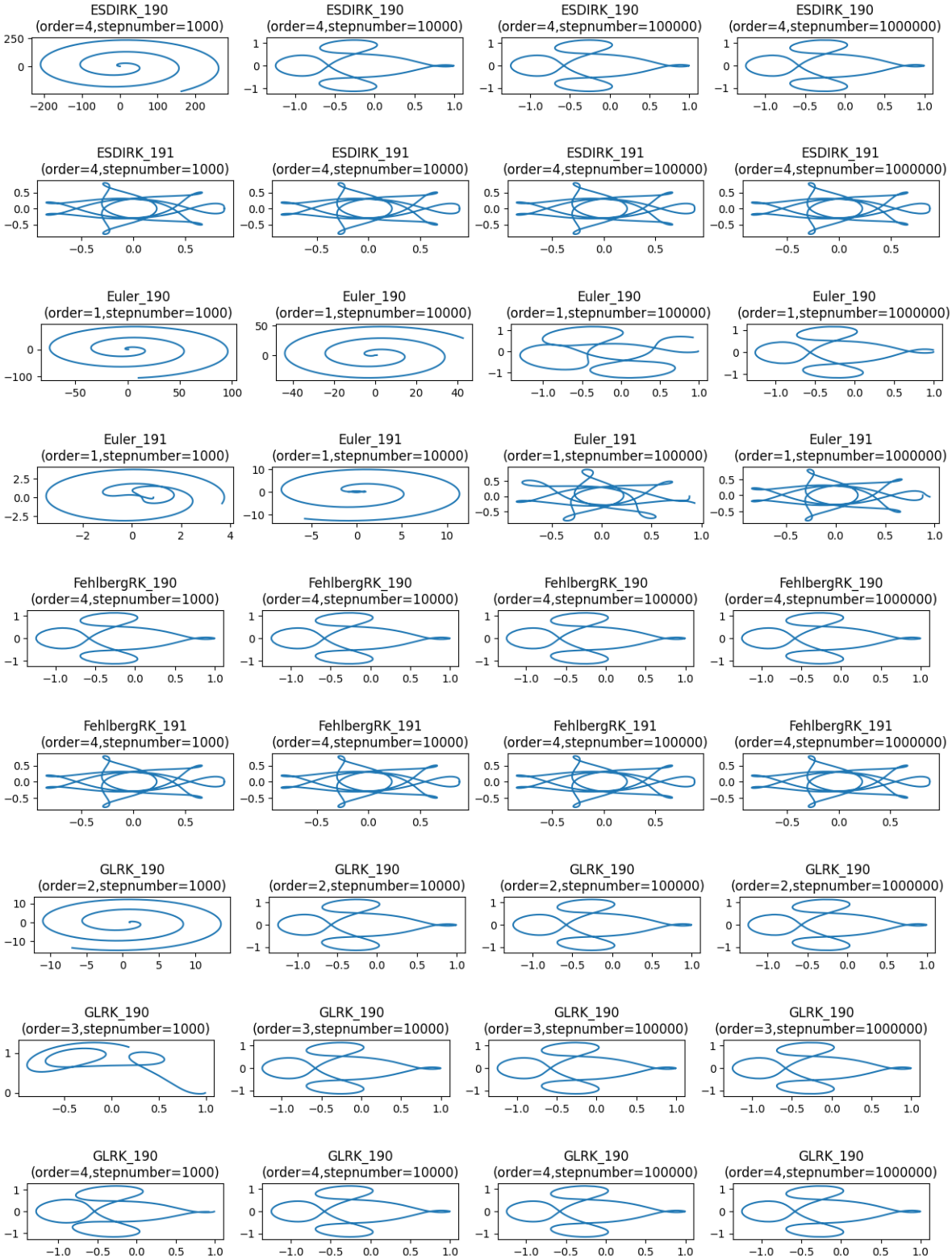
为误差算出来都很小，感觉主要误差不来自于算法）。

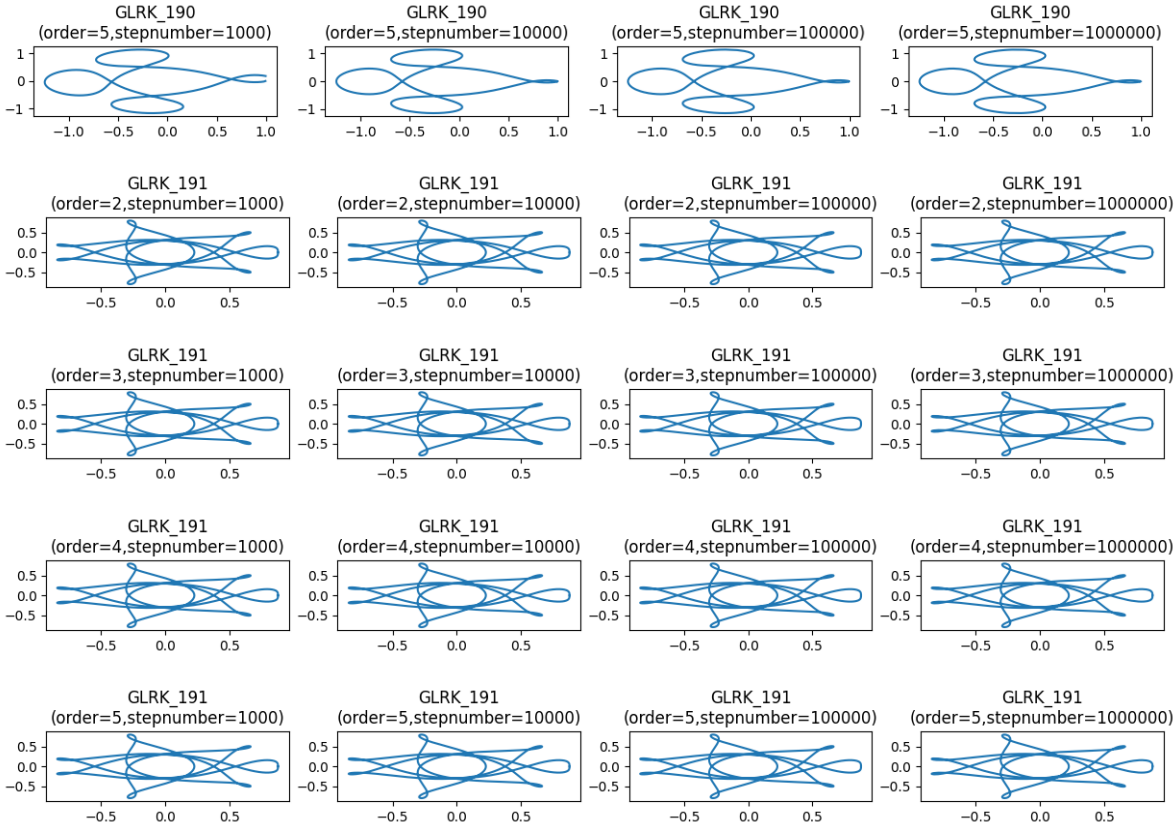
3.5 不同求解器及其不同精度的求解情况




Adams-Moulton_190
(order=2,stepnumber=1000)
Adams-Moulton_190
(order=2,stepnumber=10000)
Adams-Moulton_190
(order=2,stepnumber=100000)
Adams-Moulton_190
(order=2,stepnumber=1000000)
Adams-Moulton_190
(order=3,stepnumber=1000)
Adams-Moulton_190
(order=3,stepnumber=10000)
Adams-Moulton_190
(order=3,stepnumber=100000)
Adams-Moulton_190
(order=3,stepnumber=1000000)
Adams-Moulton_190
(order=4,stepnumber=1000)
Adams-Moulton_190
(order=4,stepnumber=10000)
Adams-Moulton_190
(order=4,stepnumber=100000)
Adams-Moulton_190
(order=4,stepnumber=1000000)
Adams-Moulton_190
(order=5,stepnumber=1000)
Adams-Moulton_190
(order=5,stepnumber=10000)
Adams-Moulton_190
(order=5,stepnumber=100000)
Adams-Moulton_190
(order=5,stepnumber=1000000)
Adams-Moulton_191
(order=2,stepnumber=1000)
Adams-Moulton_191
(order=2,stepnumber=10000)
Adams-Moulton_191
(order=2,stepnumber=100000)
Adams-Moulton_191
(order=2,stepnumber=1000000)
Adams-Moulton_191
(order=3,stepnumber=1000)
Adams-Moulton_191
(order=3,stepnumber=10000)
Adams-Moulton_191
(order=3,stepnumber=100000)
Adams-Moulton_191
(order=3,stepnumber=1000000)
Adams-Moulton_191
(order=4,stepnumber=1000)
Adams-Moulton_191
(order=4,stepnumber=10000)
Adams-Moulton_191
(order=4,stepnumber=100000)
Adams-Moulton_191
(order=4,stepnumber=1000000)
Adams-Moulton_191
(order=5,stepnumber=1000)
Adams-Moulton_191
(order=5,stepnumber=10000)
Adams-Moulton_191
(order=5,stepnumber=100000)
Adams-Moulton_191
(order=5,stepnumber=1000000)
Backward_Differentiation_190
(order=1,stepnumber=1000)
Backward_Differentiation_190
(order=1,stepnumber=10000)
Backward_Differentiation_190
(order=1,stepnumber=100000)
Backward_Differentiation_190
(order=1,stepnumber=1000000)


Backward_Differentiation_190 (order=2,stepnumber=1000)
Backward_Differentiation_190 (order=2,stepnumber=10000)
Backward_Differentiation_190 (order=2,stepnumber=100000)
Backward_Differentiation_190 (order=2,stepnumber=1000000)
Backward_Differentiation_190 (order=3,stepnumber=1000)
Backward_Differentiation_190 (order=3,stepnumber=10000)
Backward_Differentiation_190 (order=3,stepnumber=100000)
Backward_Differentiation_190 (order=3,stepnumber=1000000)
Backward_Differentiation_190 (order=4,stepnumber=1000)
Backward_Differentiation_190 (order=4,stepnumber=10000)
Backward_Differentiation_190 (order=4,stepnumber=100000)
Backward_Differentiation_190 (order=4,stepnumber=1000000)
Backward_Differentiation_191 (order=1,stepnumber=1000)
Backward_Differentiation_191 (order=1,stepnumber=10000)
Backward_Differentiation_191 (order=1,stepnumber=100000)
Backward_Differentiation_191 (order=1,stepnumber=1000000)
Backward_Differentiation_191 (order=2,stepnumber=1000)
Backward_Differentiation_191 (order=2,stepnumber=10000)
Backward_Differentiation_191 (order=2,stepnumber=100000)
Backward_Differentiation_191 (order=2,stepnumber=1000000)
Backward_Differentiation_191 (order=3,stepnumber=1000)
Backward_Differentiation_191 (order=3,stepnumber=10000)
Backward_Differentiation_191 (order=3,stepnumber=100000)
Backward_Differentiation_191 (order=3,stepnumber=1000000)
Backward_Differentiation_191 (order=4,stepnumber=1000)
Backward_Differentiation_191 (order=4,stepnumber=10000)
Backward_Differentiation_191 (order=4,stepnumber=100000)
Backward_Differentiation_191 (order=4,stepnumber=1000000)
ClassicalRK_190 (order=4,stepnumber=1000)
ClassicalRK_190 (order=4,stepnumber=10000)
ClassicalRK_190 (order=4,stepnumber=100000)
ClassicalRK_190 (order=4,stepnumber=1000000)
ClassicalRK_191 (order=4,stepnumber=1000)
ClassicalRK_191 (order=4,stepnumber=10000)
ClassicalRK_191 (order=4,stepnumber=100000)
ClassicalRK_191 (order=4,stepnumber=1000000)


ESDIRK_190
(order=4,stepnumber=1000)
ESDIRK_190
(order=4,stepnumber=10000)
ESDIRK_190
(order=4,stepnumber=100000)
ESDIRK_190
(order=4,stepnumber=1000000)
ESDIRK_191
(order=4,stepnumber=1000)
ESDIRK_191
(order=4,stepnumber=10000)
ESDIRK_191
(order=4,stepnumber=100000)
ESDIRK_191
(order=4,stepnumber=1000000)
Euler_190
(order=1,stepnumber=1000)
Euler_190
(order=1,stepnumber=10000)
Euler_190
(order=1,stepnumber=100000)
Euler_190
(order=1,stepnumber=1000000)
Euler_191
(order=1,stepnumber=1000)
Euler_191
(order=1,stepnumber=10000)
Euler_191
(order=1,stepnumber=100000)
Euler_191
(order=1,stepnumber=1000000)
FehlbergRK_190
(order=4,stepnumber=1000)
FehlbergRK_190
(order=4,stepnumber=10000)
FehlbergRK_190
(order=4,stepnumber=100000)
FehlbergRK_190
(order=4,stepnumber=1000000)
FehlbergRK_191
(order=4,stepnumber=1000)
FehlbergRK_191
(order=4,stepnumber=10000)
FehlbergRK_191
(order=4,stepnumber=100000)
FehlbergRK_191
(order=4,stepnumber=1000000)
GLRK_190
(order=2,stepnumber=1000)
GLRK_190
(order=2,stepnumber=10000)
GLRK_190
(order=2,stepnumber=100000)
GLRK_190
(order=2,stepnumber=1000000)
GLRK_190
(order=3,stepnumber=1000)
GLRK_190
(order=3,stepnumber=10000)
GLRK_190
(order=3,stepnumber=100000)
GLRK_190
(order=3,stepnumber=1000000)
GLRK_190
(order=4,stepnumber=1000)
GLRK_190
(order=4,stepnumber=10000)
GLRK_190
(order=4,stepnumber=100000)
GLRK_190
(order=4,stepnumber=1000000)



注意 GLRK 图片中标示为 order 的实际上是 state number，order 是它的两倍。

自适应步长的算法，stepnumber 指的是一开始输入的 stepnumber，即只影响第一步的 stepsize。

Chapter 4

Conclusions

结合 log 文件和 3.3 节，计算速度快又计算准确的是自适应步长的方法，其次是 RK 方法，再其次是 LMM 方法。对于固定步长的方法，隐式的方法比显式的要慢（程序中设定了不动点迭代超过一万次时，若前后两项的误差还没有小于机器精度，则停止迭代），同一求解器高阶所用的时间变长，但是计算准确率大幅提高。

当误差很小时，从 3.4 max-norm 可以看出加大步长，误差减小并不符合精度，推测是此时算法的误差不占主要地位。当误差很大时，绘制的折线也不符合，结合 3.5 节可知是因为算法并没有较为准确的绘制图像。

在启动 LMM 算法前，为了获取足够的数据，我一开始使用 Euler 方法，效果非常差，后来使用了 ClassicalRK 方法，得到明显改观，这也体现出要求解的两个问题对初始条件非常敏感。

程序设计的不足之处在于，

- 一开始没有认识到其实所有求解器的主要不同在于如何从上一个 step 的数据得到下一个 step 的数据，导致有许多的重复代码。
- 对于自适应步长的方法如何设定其 $E_{abs}, E_{rel}, \rho, \rho_{max}, \rho_{min}$ 没有好的想法，直接硬编码到其中，给与之相关的测试带来麻烦。
- 对于确定自适应步长算法确定满足误差小于 0.001 的最小 cpu time 的参数，没有设计好的算法。

- 对于控制自适应步长算法的总步数，没有设计好的算法，采用了直接遍历区间的方法，并且找到的参数绘制出来的图片效果不理想，可能存在更好的参数。